

SQL Joins Practice: 10 Queries

Concepts covered: INNER JOIN, LEFT JOIN, multi-table joins, JOIN + GROUP BY | Dataset: Northwind-style (7 tables) | 1 October 2026

Objective

Combine the 7 tables cleaned in earlier projects using INNER JOIN and LEFT JOIN, and practice the difference between them. This project also surfaced a real data-quality issue that the earlier single-table projects didn't: see section 2 before reading the query outputs.

Data quality note: read this first

Table row counts: Customers 23, Orders 21, Products 25, OrderDetails 26. Checking the foreign keys shows: **only 3 of 21 orders have a CustomerID that matches a row in Customers**, and **only 9 of 26 order lines have a ProductID that matches a row in Products**. The other IDs (e.g. CustomerID 90, 81, 34... and ProductID up to 72) don't exist in those tables.

This means an INNER JOIN between Orders and Customers, or between OrderDetails and Products, keeps only a small fraction of the rows — not because the query is wrong, but because the source data has gaps. Each affected query below is marked with a note explaining exactly how many rows this costs.

Queries and outputs

Query 1: Orders with customer name (INNER JOIN)

```
SELECT o.OrderID, c.CustomerName, o.OrderDate
FROM Orders o
INNER JOIN Customers c ON o.CustomerID = c.CustomerID
ORDER BY o.OrderDate;
```

■ Only 3 of 21 orders have a CustomerID that exists in Customers, so INNER JOIN drops 18 rows here.

OrderID	CustomerName	OrderDate
10255	Feinkost Käfer	2022-07-12
10257	Tokyo Delicacies	2022-07-14
10258	Eurasian Delicacies	2022-07-15

3 row(s) returned.

Query 2: Products with category name (INNER JOIN)

```
SELECT p.ProductName, cat.CategoryName, p.Price
FROM Products p
INNER JOIN Categories cat ON p.CategoryID = cat.CategoryID
ORDER BY cat.CategoryName;
```

ProductName	CategoryName	Price
Chais	Beverages	18.00
Chang	Beverages	19.00
Apple Juice	Beverages	14.00
Green Tea	Beverages	12.00
Instant Coffee	Beverages	9.50
Vegetable Stir-Fry Mix	Beverages	19.99

ProductName	CategoryName	Price
Clam Chowder	Beverages	22.00
Aniseed Syrup	Condiments	10.00
Chef Anton's Cajun Seasoning	Condiments	22.00
Chef Anton's Gumbo Mix	Condiments	21.35
Tomato Sauce	Condiments	15.00
Chocolate Sauce	Condiments	18.00
Basil Pesto	Condiments	7.20
Sesame Oil	Condiments	11.00
Peanut Butter	Condiments	15.00

25 row(s) returned (showing first 15 of 25).

Query 3: Products with supplier name (INNER JOIN)

```
SELECT p.ProductName, s.SupplierName, p.Price
FROM Products p
INNER JOIN Suppliers s ON p.SupplierID = s.SupplierID
ORDER BY s.SupplierName;
```

ProductName	SupplierName	Price
Jasmine Rice	Cooperativa de Quesos 'Las Cabras'	25.00
Clam Chowder	Cooperativa de Quesos 'Las Cabras'	22.00
Mango Chutney	Cooperativa de Quesos 'Las Cabras'	9.95
Maple Syrup	Cooperativa de Quesos 'Las Cabras'	25.00
Almond Flour	Cooperativa de Quesos 'Las Cabras'	11.50
Granola Bars	Cooperativa de Quesos 'Las Cabras'	16.00
Vanilla Extract	Cooperativa de Quesos 'Las Cabras'	30.00
Chais	Exotic Liquids	18.00
Chang	Exotic Liquids	19.00
Aniseed Syrup	Exotic Liquids	10.00
Apple Juice	Exotic Liquids	14.00
Green Tea	Exotic Liquids	12.00
Instant Coffee	Exotic Liquids	9.50
Cream Cheese	Grandma Kelly's Homestead	2.70
Basil Pesto	Grandma Kelly's Homestead	7.20

25 row(s) returned (showing first 15 of 25).

Query 4: Order lines with product names (INNER JOIN)

```
SELECT od.OrderID, p.ProductName, od.Quantity
FROM OrderDetails od
INNER JOIN Products p ON od.ProductID = p.ProductID
ORDER BY od.OrderID;
```

■ Only 9 of 26 order lines have a ProductID that exists in Products, so 17 rows are dropped.

OrderID	ProductName	Quantity
10248	Chocolate Sauce	12
10249	Basil Pesto	9
10250	Vanilla Extract	8
10251	Jasmine Rice	15
10254	Vegetable Stir-Fry Mix	14
10256	Whole Wheat Pasta	18
10257	Granola Bars	4
10261	Cream Cheese	16
10265	Instant Coffee	14

9 row(s) returned.

Query 5: Orders with shipper name (INNER JOIN)

```
SELECT o.OrderID, o.OrderDate, s.CompanyName AS Shipper
FROM Orders o
INNER JOIN Shippers s ON o.ShipperID = s.ShipperID
ORDER BY o.OrderDate;
```

OrderID	OrderDate	Shipper
10248	2022-07-04	Federal Shipping
10249	2022-07-05	Speedy Express
10250	2022-07-08	United Package
10251	2022-07-08	Speedy Express
10252	2022-07-09	United Package
10253	2022-07-10	Federal Shipping
10254	2022-07-11	Speedy Express
10255	2022-07-12	United Package
10256	2022-07-13	Speedy Express
10257	2022-07-14	United Package
10258	2022-07-15	Federal Shipping
10259	2022-07-16	Speedy Express
10260	2022-07-17	Federal Shipping
10261	2022-07-18	Speedy Express
10262	2022-07-19	United Package

21 row(s) returned (showing first 15 of 21).

Query 6: Every customer, with orders if any (LEFT JOIN)

```
SELECT c.CustomerName, o.OrderID, o.OrderDate
FROM Customers c
LEFT JOIN Orders o ON c.CustomerID = o.CustomerID
ORDER BY c.CustomerName;
```

■ All 23 customers are kept (LEFT JOIN), but most show NULL order columns since they have no matching order.

CustomerName	OrderID	OrderDate
Alfreds Futterkiste	NULL	NULL
Ana Trujillo Emparedados y helados	NULL	NULL
Antonio Moreno Taquería	NULL	NULL
Blauer See Delikatessen	NULL	NULL
Blondel père et fils	NULL	NULL
Bon app'	NULL	NULL
Chocolaterie Richart	NULL	NULL
Cornershop	NULL	NULL
Delicias de España	NULL	NULL
Dolores's Fine Foods	NULL	NULL
Eurasian Delicacies	10258	2022-07-15
Feinkost Käfer	10255	2022-07-12
Fiesta Foods	NULL	NULL
Forno di Napoli	NULL	NULL
Gastronomia S.r.l.	NULL	NULL

23 row(s) returned (showing first 15 of 23).

Query 7: Customers with no matching orders (LEFT JOIN + IS NULL)

```
SELECT c.CustomerID, c.CustomerName
FROM Customers c
LEFT JOIN Orders o ON c.CustomerID = o.CustomerID
WHERE o.OrderID IS NULL;
```

■ 20 of 23 customers have no matching order at all — a side effect of the CustomerID mismatch, not necessarily real 'customers with zero orders'.

CustomerID	CustomerName
1	Alfreds Futterkiste
2	Ana Trujillo Emparedados y helados
3	Antonio Moreno Taquería
4	Blauer See Delikatessen
5	Blondel père et fils
6	Bon app'
7	Chocolaterie Richart
8	Cornershop
9	Delicias de España
10	Dolores's Fine Foods
13	Fiesta Foods

CustomerID	CustomerName
14	Forno di Napoli
15	Gastronomia S.r.l.
16	Gourmet Delights
17	Jamaica Imports

20 row(s) returned (showing first 15 of 20).

Query 8: Every product, with order quantities if ordered (LEFT JOIN)

```
SELECT p.ProductName, od.OrderID, od.Quantity
FROM Products p
LEFT JOIN OrderDetails od ON p.ProductID = od.ProductID
ORDER BY p.ProductName;
```

ProductName	OrderID	Quantity
Almond Flour	NULL	NULL
Aniseed Syrup	NULL	NULL
Apple Juice	NULL	NULL
Basil Pesto	10249	9
Chais	NULL	NULL
Chang	NULL	NULL
Chef Anton's Cajun Seasoning	NULL	NULL
Chef Anton's Gumbo Mix	NULL	NULL
Chocolate Sauce	10248	12
Clam Chowder	NULL	NULL
Cream Cheese	10261	16
Granola Bars	10257	4
Green Tea	NULL	NULL
Grilled Chicken	NULL	NULL
Instant Coffee	10265	14

25 row(s) returned (showing first 15 of 25).

Query 9: Order lines with customer and product names (3-table JOIN)

```
SELECT o.OrderID, c.CustomerName, p.ProductName, od.Quantity
FROM Orders o
INNER JOIN Customers c ON o.CustomerID = c.CustomerID
INNER JOIN OrderDetails od ON o.OrderID = od.OrderID
INNER JOIN Products p ON od.ProductID = p.ProductID
ORDER BY o.OrderID;
```

■ Only 1 row survives all three joins, since it needs a matching CustomerID AND a matching ProductID at the same time.

OrderID	CustomerName	ProductName	Quantity
10257	Tokyo Delicacies	Granola Bars	4

1 row(s) returned.

Query 10: Total revenue per product (JOIN + GROUP BY)

```
SELECT p.ProductName, SUM(od.Quantity * p.Price) AS total_revenue
FROM OrderDetails od
INNER JOIN Products p ON od.ProductID = p.ProductID
GROUP BY p.ProductName
ORDER BY total_revenue DESC;
```

■ Only the 9 order lines with a valid ProductID contribute to revenue; totals for the other 17 lines are not reflected here.

ProductName	total_revenue
Jasmine Rice	375.00
Vegetable Stir-Fry Mix	279.86
Vanilla Extract	240.00
Chocolate Sauce	216.00
Instant Coffee	133.00
Basil Pesto	64.80
Granola Bars	64.00
Whole Wheat Pasta	63.00
Cream Cheese	43.20

9 row(s) returned.

What I noticed testing each join type

- **INNER JOIN** only keeps rows with a match on both sides — when the foreign keys don't line up, rows disappear silently, with no error. That's why checking row counts before and after a join matters.
- **LEFT JOIN** keeps every row from the left table regardless of a match, filling the right-hand columns with NULL when there's nothing to join to.
- **WHERE right_table.column IS NULL** after a LEFT JOIN is the standard way to find rows with no match — used in query 7 to find customers with no orders.
- Chaining three INNER JOINS multiplies the mismatch problem: a row only survives if every join in the chain finds a match, which is why query 9 returns just 1 row here.

Interview questions

INNER JOIN vs LEFT JOIN? INNER JOIN returns only rows that have a match in both tables. LEFT JOIN returns every row from the left (first) table, with NULLs in the right table's columns where no match exists.

Why use IS NULL after a LEFT JOIN? After a LEFT JOIN, unmatched left-table rows get NULL in every column that came from the right table. Filtering on WHERE right_table.column IS NULL isolates exactly those unmatched rows — for example, customers who never placed an order.